



RDBMS to MongoDB Day

[City] — [Month] [Day], [Year]

Schedule	
9:00 AM - 9:15 AM	Welcome and Introduction
9:15 AM - 12:15 PM	Data Modeling for RDBMS Professionals
12:15 PM - 1:15 PM	Lunch Provided by MongoDB
1:15 PM - 2:45 PM	Moving Data to MongoDB with Relational Migrator
2:45 PM - 3:00 PM	Break
3:00 PM - 4:45 PM	From SQL to the MongoDB Query API
4:45 PM - 5:00 PM	Closing Notes
5:00 PM	Happy Hour with MongoDB Experts



All resources, slides, and self-paced content can be found at
[https://mdb.link/devday-\[CITY\]-25](https://mdb.link/devday-[CITY]-25)



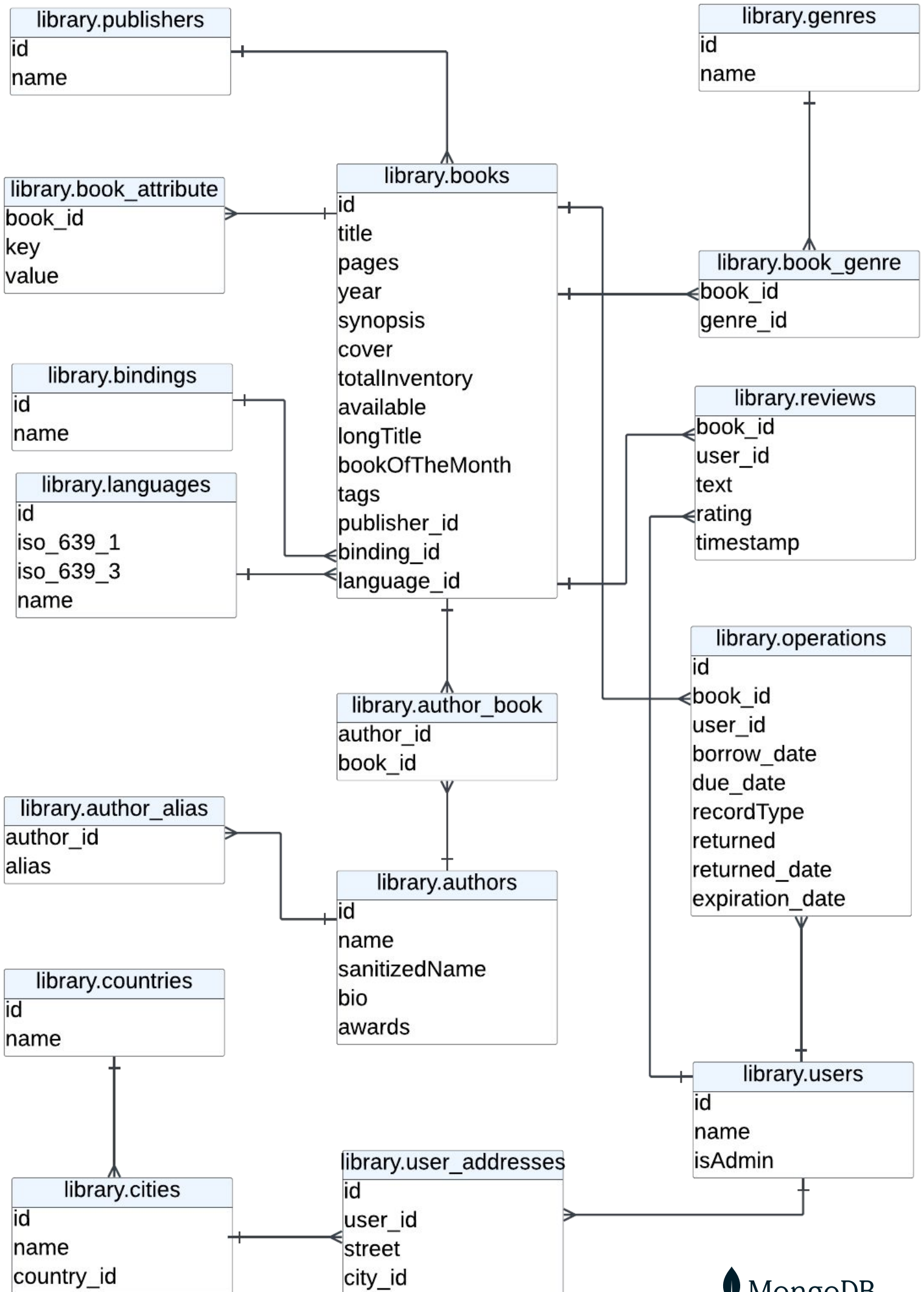
RDBMS to MongoDB Day

[City] — [Month] [Day], [Year]

Schedule	
9:00 AM - 9:30 AM	Welcome and Introduction
9:30 AM - 11:00 AM	Data Modeling for RDBMS Professionals
11:00 AM - 11:20 AM	Coffee Break
11:20 AM - 1:00 PM	(Cont.) Data Modeling for RDBMS Professionals
1:00 PM - 1:45 PM	Lunch Provided by MongoDB
1:45 PM - 3:30 PM	Moving Data to MongoDB with Relational Migrator
3:30 PM - 3:45 PM	Break
3:45 PM - 5:15 PM	From SQL to the MongoDB Query API
5:15 PM - 5:30 PM	Closing Notes
5:30 PM	Happy Hour with MongoDB Experts



All resources, slides, and self-paced content can be found at
[https://mdb.link/devday-\[CITY\]-25](https://mdb.link/devday-[CITY]-25)



Functional Requirements

The system should allow **readers** to:

- R1. Browse a catalogue of library books.
- R2. View the extended details of a book — including title, author, and synopsis — and reviews from other readers.
- R3. Make a book reservation if the book is available. Book reservations expire after 12 hours.
- R4. See a list of their unexpired reservations.
- R5. See a list of all books they have borrowed from the library and their due dates.
- R6. View the details for an author, including all of the books they have published.

The system should allow **librarians** to:

- L1. See a list of all book reservations, including their expiration time.
- L2. See a list of all borrowed books, including their due date.
- L3. Filter reservations and borrowed books by user.
- L4. Lend books to readers.
- L5. Mark books as returned.

Library Data

- A. The library has **10k active readers weekly**.
- B. The library has about **250k books**.
- C. Readers will consult the library website on a **weekly basis** and borrow books **every week**.
- D. Most readers **reserve** and **borrow** up to **3 books** every week.
- E. **Reservations** can be made online, but books can only be **borrowed** on-site.
- F. Readers rarely read more than **5 book reviews**.
- G. Readers browse about **25 books** before reserving.
- H. Readers leave **reviews for 1 in 5 books** they read.

Performance Requirements

- P1. The book details page that shows book availability and recent reviews should load in less than **300ms**.
- P2. Readers should be able to reserve a book in less than **3 seconds**.
- P3. Librarians should be able to lend a book or mark a book as returned in less than **3 seconds**.
- P4. Readers should be able to see the list with all of their reservations and borrowed books in less than **500ms**.

Identify Tables

Tables

Classify tables by their type: Entity, Lookup, Associative, or Weak

Table Name	Type

Worksheet Page 1: Tables

Table types

- **Entity:** strong business entities, could potentially exist alone.
- **Lookup:** dimension values for attributes in other tables
- **Associative:** contain references to resolve many-to-many relationships
- **Weak:** many-to-one data subordinate to a parent table

Embedding vs referencing guidelines

Guideline name	Question	Embed	Reference
Simplicity	Would keeping the pieces of information together lead to a simpler data model and code?	Yes	
Go together	Do the pieces of information express a "has-a," "contains," or similar relationship?	Yes	
Query atomicity	Does the application query the pieces of information together?	Yes	
Update complexity	Are the pieces of information updated together?	Yes	
Archival	Should the pieces of information be archived at the same time?	Yes	
Cardinality	Is there a high cardinality (current or growing) in a "many" side of the relationship?	No	Yes
Data duplication	Would data duplication be too complicated to manage and undesired?	No	Yes
Document size	Would the combined size of the pieces of information take too much memory or transfer bandwidth for the application?	No	Yes
Document growth	Would the embedded piece grow without bound?	No	Yes
Workload	Are the pieces of information written at different times in a write-heavy workload?		Yes
Individuality	For the children side of the relationship, can the pieces exist by themselves without a parent?		Yes

Schema design anti-patterns

1. Unbounded arrays
2. Bloated documents
3. Massive number of collections
4. Unnecessary indexes
5. Separating data that is accessed together
6. Case-insensitive queries without case-insensitive indexes



From Relational to Document Model

Data Modeling for MongoDB

With MongoDB, you can design the most optimal schema for your use case instead of a generic schema for any use case.

Core MongoDB Design Concepts



Embedding

You can store documents within other documents to represent relationships and complex hierarchies.

Embedding allows for storing related data within a single document, unlike relational databases that use separate tables and JOINS to represent relationships.



Polymorphism

You can store documents with different fields within the same collection.

Polymorphism allows similar types of data to be stored in the same collection and retrieved together.

Polymorphism also allows you to evolve your schema without downtime. Different versions of the schema can coexist.



Arrays

The value of a field in a document can be an array of values, which can themselves be documents.

Arrays enable many data modeling options, particularly for one-to-many and many-to-many relationships, allowing you to store related data together.

When modeling
data for MongoDB,
your workload
should drive your
data model.

Schema Validation

Highly recommended for maintaining data integrity

Defined at the collection level

Enforced by the database
regardless of the connection source

```
{
  bsonType: 'object',
  required: ['name', 'biography'],
  properties: {
    name: { bsonType: 'string' },
    biography: { bsonType: 'string' }
  },
  aliases: {
    bsonType: 'array',
    maxItems: 10
  }
};
```

```
{
  bsonType: 'object',
  required: [
    'rating',
    'reviewer',
    'book'
  ],
  properties: {
    text: { bsonType: 'string' },
    rating: {
      bsonType: 'int',
      minimum: 1,
      maximum: 5
    },
    reviewer: { bsonType: 'objectId' },
    book: { bsonType: 'objectId' },
  }
};
```

Modeling Relationships

One-to-one.

Embed in the natural parent entity.

Exception: Avoid embedding if it causes bloated documents, leading to high memory or bandwidth usage.

One-to-few.

Embed on the "one" side. Exceptions:

- If the "few" are frequently accessed on their own.
- Avoid embedding if it causes bloated documents, leading to high memory or bandwidth usage.

One-to-many (medium cardinality).

Follow the "Embedding vs referencing guidelines" sheet. Most importantly, store together if accessed together.

One-to-zillions (unbounded growth).

Use references on the "many" side.

Many-to-many.

Reference on the more often queried side.

Exception: If the array with references could grow unbounded, reference from the other side instead.



Schema Design Patterns and Anti-Patterns

Data Modeling for MongoDB

Use schema design patterns to optimize your data model based on how your application queries and uses data. Avoid anti-patterns that can degrade performance.

Patterns

Extended reference

An "in-between" solution between embedding and referencing for minimizing the number of joins for frequent operations

Use it when data is stored in separate collections that need to be joined frequently

Computed

Precompute and store values, offsetting the computation on write instead of read

Addresses the case where computational tasks are performed frequently on data retrieval

Use it on high read-to-write ratio on documents where computation is needed

Inheritance

Simplify the retrieval of similar data by storing it in a polymorphous collection and using a field to distinguish between the types

Addresses the issue with documents that are similar but still have distinct features; allows simpler retrieval of data as it can coexist

Anti-Patterns

Unbounded arrays

Large growing arrays with an unlimited number of elements

Use referencing on the "many" side or embed only a subset of the array

Bloated documents

Oversized documents containing data that isn't accessed together

They inflate the working set, potentially exceeding the WiredTiger internal cache and degrading performance

“ MongoDB design patterns are reusable solutions for many of the commonly occurring use cases encountered when designing applications that leverage persistence in MongoDB.

Daniel Coupal
Senior Staff Developer Advocate,
MongoDB



Learn more about MongoDB



MongoDB Skill Badges

Learn and earn free skill badges to build your MongoDB expertise in areas like data modeling, performance, and sharding. Each badge is a Credly-certified digital credential you can add to your LinkedIn profile. Start learning today and earn a badge in 60 - 90 minutes.

<https://mdb.link/skills>



MongoDB Articles & Tutorials

Elevate your development skills and efficiency with MongoDB articles and tutorials. Check out our content on Medium to discover insights about using MongoDB. Dive into the technical details in our tutorials on DEV.

<https://medium.com/mongodb>

<https://dev.to/mongodb>



Join Your Local MongoDB Community

Learn from community members, build together, network and be inspired.

Explore our community.

<https://mdb.link/mongodb-usergroups>



Learn more about MongoDB

MongoDB Skill Badges

Learn and earn free **skill badges** to build your **MongoDB expertise** in areas like data modeling, performance, and sharding. Each badge is a Credly-certified **digital credential** you can add to your LinkedIn profile. Start learning today and earn a badge in 60 - 90 minutes.

<https://mdb.link/skills>

MongoDB Articles & Tutorials

Elevate your development skills and efficiency with MongoDB **articles and tutorials**. Check out our content on Medium to **discover insights** about using MongoDB. Dive into the **technical details** in our tutorials on DEV.

<https://medium.com/mongodb>

<https://dev.to/mongodb>

Join Your Local MongoDB Community

Learn from **community members**, build **together**, network and be **inspired**.

Explore our community.

<https://mdb.link/mongodb-usergroups>